JAVA PROGRAMMING / DSA

Chapter 31: Backtracking in Java

Personal Notes

Some problems don't have a clever formula — you have to TRY every possibility. Generating all permutations of a list. Filling a Sudoku. Placing 8 queens on a chessboard so none attack each other. Finding a path through a maze.

These are EXPLORATION problems, and backtracking is the technique that handles them. It's recursion with a twist: when a partial solution leads to a dead end, you UNDO your last step and try something else. "Take a step → if it doesn't work, take it back → try another step." That's it. The rest is just bookkeeping.

1. What is Backtracking?

Backtracking is a refinement of recursion for problems that involve choosing a SEQUENCE OF DECISIONS. At each step you pick an option, recurse, and if the recursion fails (or you've enumerated all possibilities from there), you UNDO your pick and try the next option.

The Pattern in Three Sentences

- CHOOSE — pick an option for the current step
- EXPLORE — recurse into the next step with that choice
- UN-CHOOSE — undo your pick before trying the next option

```
void backtrack(state):
    if goal reached:
        record/print solution
        return

    for each choice at this step:
        if choice is valid:
            apply choice            ← CHOOSE
            backtrack(new state)    ← EXPLORE
            undo choice              ← UN-CHOOSE
```

> **Mental model:** Backtracking explores a TREE of possibilities. The root is "nothing chosen yet." Each branch is one choice. Each leaf is either a complete solution or a dead end. Backtracking is a DFS through this tree, with the ability to abandon a branch as soon as it's known to fail.

2. Why Backtracking Beats Brute Force

Brute force generates EVERY possibility and checks each one at the end. Backtracking checks AS IT GOES — abandoning partial solutions the moment they become invalid. The savings can be massive.

Example: Placing Queens on a 4×4 board

```
Brute force:  4^4 = 256 placements, then check each → 256 checks

Backtracking: ~17 placements explored — stops as soon as two
              queens would attack each other
```

The deeper the search tree, the bigger the win. For N-Queens on an 8×8 board, brute force is 16 million possibilities; backtracking explores about 15,720.

The Pruning Idea

```
            [start]
            /  |  \
           A   B   C
          /|\  X*  /|\
         ...       ...

If branch B leads to no valid solution,
we PRUNE the entire B subtree — saves all that work.

*X = pruned
```

3. First Example — Print All Subsets

Given an array, print every possible subset (the POWER SET). For [1, 2, 3] there are 2^3 = 8 subsets including the empty one.

The Choice at Each Step

For every element, you have exactly 2 choices: INCLUDE it in the subset, or EXCLUDE it. The decision tree has depth N and 2^N leaves.

```
[1,2,3] → all subsets

                    [ ]
              exclude/    \include 1
                    /      \
                 [ ]        [1]
           exc/ \inc   exc/ \inc 2
             /     \     /     \
           [ ]     [2]  [1]    [1,2]
      ... ... (similarly for choice 3)
```

The Code

```
import java.util.*;

public class Main {
    public static void subsets(int[] arr, int i, List<Integer> current) {
        if (i == arr.length) {
```

```
            System.out.println(current);
            return;
        }

        // Choice 1: EXCLUDE arr[i]
        subsets(arr, i + 1, current);

        // Choice 2: INCLUDE arr[i]
        current.add(arr[i]);                        // CHOOSE
        subsets(arr, i + 1, current);               // EXPLORE
        current.remove(current.size() - 1);         // UN-CHOOSE
    }

    public static void main(String[] args) {
        subsets(new int[]{1, 2, 3}, 0, new ArrayList<>());
    }
}

// Output:
// []
// [3]
// [2]
// [2, 3]
// [1]
// [1, 3]
// [1, 2]
// [1, 2, 3]
```

Notice the un-choose step: current.remove(current.size()-1) is the BACKTRACK part. Without it, the "current" list would keep growing forever, and you'd never explore alternatives. Every "choose" needs a matching "un-choose."

4. The Universal Backtracking Template

Almost every backtracking problem fits this exact shape. Once you internalize it, you can adapt it to any of them.

```
    void backtrack(state, choicesSoFar) {

        // 1. BASE CASE — solution found
        if (isComplete(state)) {
            record(choicesSoFar);
            return;
        }

        // 2. TRY EACH OPTION at this step
        for (each option at current step) {
```

```
            // 3. PRUNE — skip invalid options early
            if (!isValid(option)) continue;

            // 4. CHOOSE
            choicesSoFar.add(option);

            // 5. EXPLORE
            backtrack(updatedState, choicesSoFar);

            // 6. UN-CHOOSE
            choicesSoFar.remove(last);
        }
    }
```

Five steps to write any backtracking solution:

1. Define the STATE — what data represents "where I am" right now?
2. Define the BASE CASE — when have I built a complete solution?
3. Define the CHOICES — what options exist at this step?
4. Define the PRUNING — which choices can I skip immediately as invalid?
5. Define CHOOSE / UN-CHOOSE — how do I apply and undo a choice?

5. Classic Example — Permutations

Generate ALL orderings of [1, 2, 3]. Expected: 6 permutations (3! = 6).

Strategy

- State — current permutation being built, and a "used" array tracking which numbers are already used
- Base case — current.size() == arr.length (we placed every element)
- Choices — any unused number
- Pruning — skip numbers already used

Code

```
public static void permute(int[] arr, boolean[] used, List<Integer>
current) {
    if (current.size() == arr.length) {
        System.out.println(current);
        return;
    }

    for (int i = 0; i < arr.length; i++) {
        if (used[i]) continue;          // PRUNE — skip used

        used[i] = true;                  // CHOOSE
        current.add(arr[i]);
```

```
        permute(arr, used, current);    // EXPLORE
        current.remove(current.size() - 1);   // UN-CHOOSE
        used[i] = false;
    }
}

// Call:
permute(new int[]{1, 2, 3}, new boolean[3], new ArrayList<>());

// Output:
// [1, 2, 3]
// [1, 3, 2]
// [2, 1, 3]
// [2, 3, 1]
// [3, 1, 2]
// [3, 2, 1]
```

Note the symmetry between CHOOSE and UN-CHOOSE — they're exact mirror operations. Every modification you make before recursing must be reversed after.

6. N-Queens — The Backtracking Classic

Place N queens on an N×N chessboard so no two queens attack each other (same row, same column, or same diagonal). For N=4, there are 2 valid solutions.

Strategy

- Place one queen per ROW (this guarantees no row conflicts)
- For each row, try every column
- Before placing, check that the column and both diagonals are clear of earlier queens
- If no valid column exists in this row, BACKTRACK to the previous row and try a different column

Code

```
public class NQueens {
    static int count = 0;

    public static void solve(int n) {
        int[] queens = new int[n];   // queens[row] = column
        place(queens, 0, n);
        System.out.println("Total solutions: " + count);
    }

    static void place(int[] queens, int row, int n) {
        if (row == n) {
            count++;
            print(queens, n);
            return;
        }
```

```
        for (int col = 0; col < n; col++) {
            if (isSafe(queens, row, col)) {
                queens[row] = col;          // CHOOSE
                place(queens, row + 1, n);  // EXPLORE
                // UN-CHOOSE is implicit — queens[row] will be overwritten
next loop
            }
        }
    }

    static boolean isSafe(int[] queens, int row, int col) {
        for (int r = 0; r < row; r++) {
            int c = queens[r];
            if (c == col) return false;                       // same
column
            if (Math.abs(c - col) == Math.abs(r - row))
                return false;                                  // same
diagonal
        }
        return true;
    }

    static void print(int[] queens, int n) {
        for (int r = 0; r < n; r++) {
            StringBuilder sb = new StringBuilder();
            for (int c = 0; c < n; c++) {
                sb.append(queens[r] == c ? "Q " : ". ");
            }
            System.out.println(sb);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        solve(4);
    }
}
```

Output for N=4

```
 . Q . .          . . Q .
 . . . Q          Q . . .
 Q . . .          . . . Q
 . . Q .          . Q . .

 Total solutions: 2
```

> **Why this is fast:** Without pruning, we'd try N^N placements. With backtracking, the moment a queen attacks an earlier one, we abandon that branch — entire subtrees of bad placements never get explored. For N=8, this drops 16 million to about 15,720 — a ~1000× speedup.

7. Rat in a Maze

Given an N×N grid with some cells blocked, find a path from (0,0) to (N-1,N-1), moving only down or right. Print all valid paths.

Strategy

- At each cell, try all valid moves (down, right — or all 4 directions if allowed)
- Mark the cell as visited
- Recurse to the next cell
- Unmark on the way back (the backtrack step)

Code

```
public class RatInMaze {
    static void solve(int[][] maze, int n) {
        int[][] sol = new int[n][n];
        findPath(maze, 0, 0, sol, n);
    }

    static void findPath(int[][] maze, int r, int c, int[][] sol, int n) {
        // BASE CASE — reached the goal
        if (r == n - 1 && c == n - 1) {
            sol[r][c] = 1;
            print(sol, n);
            sol[r][c] = 0;
            return;
        }

        // PRUNE — out of bounds or blocked or already visited
        if (r < 0 || r >= n || c < 0 || c >= n) return;
        if (maze[r][c] == 0) return;
        if (sol[r][c] == 1) return;

        sol[r][c] = 1;                  // CHOOSE
        findPath(maze, r + 1, c, sol, n);   // try down
        findPath(maze, r, c + 1, sol, n);   // try right
        sol[r][c] = 0;                   // UN-CHOOSE
    }

    static void print(int[][] sol, int n) {
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) System.out.print(sol[r][c] + " ");
```

```
            System.out.println();
        }
        System.out.println();
    }
}
```

8. Sudoku Solver

Fill empty cells in a 9×9 grid so every row, column, and 3×3 sub-box contains digits 1–9 exactly once. Classic backtracking — try each digit in each empty cell, undo if it leads nowhere.

The Strategy

- Find the next empty cell
- Try each digit from 1 to 9
- If a digit is valid (no conflict in row, column, or box), place it and recurse
- If recursion fails, undo the placement and try the next digit
- If no digit works, return false — backtrack further

Code

```
public class Sudoku {
    public static boolean solve(int[][] board) {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == 0) {            // empty cell
                    for (int d = 1; d <= 9; d++) {
                        if (isValid(board, r, c, d)) {
                            board[r][c] = d;         // CHOOSE
                            if (solve(board)) return true;   // EXPLORE
                            board[r][c] = 0;          // UN-CHOOSE
                        }
                    }
                    return false;   // no digit worked — backtrack
                }
            }
        }
        return true;   // all cells filled
    }

    static boolean isValid(int[][] b, int r, int c, int d) {
        for (int i = 0; i < 9; i++) {
            if (b[r][i] == d) return false;   // row
            if (b[i][c] == d) return false;   // column
        }
        int boxR = (r / 3) * 3, boxC = (c / 3) * 3;
        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
```

```
                if (b[boxR + i][boxC + j] == d) return false;   // 3x3 box
        return true;
    }
}
```

Why this works: Sudoku has up to 9 choices per cell and up to ~50 empty cells in a hard puzzle — 9^50 is astronomical. Backtracking with the simple "valid" check prunes most of that. Solving a real puzzle takes milliseconds, even though the search space is bigger than the number of atoms in the universe.

9. Word Search in a Grid

Given a grid of letters and a word, check if the word can be formed by tracing adjacent cells (up, down, left, right), never reusing a cell. Classic 2D backtracking.

```
public class WordSearch {
    public static boolean exists(char[][] grid, String word) {
        for (int r = 0; r < grid.length; r++)
            for (int c = 0; c < grid[0].length; c++)
                if (dfs(grid, r, c, word, 0)) return true;
        return false;
    }

    static boolean dfs(char[][] grid, int r, int c, String word, int i) {
        if (i == word.length()) return true;   // found all letters
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length)
            return false;
        if (grid[r][c] != word.charAt(i)) return false;

        char saved = grid[r][c];
        grid[r][c] = '#';                          // mark visited
(CHOOSE)

        boolean found = dfs(grid, r + 1, c, word, i + 1)
                     || dfs(grid, r - 1, c, word, i + 1)
                     || dfs(grid, r, c + 1, word, i + 1)
                     || dfs(grid, r, c - 1, word, i + 1);

        grid[r][c] = saved;                         // UN-CHOOSE
        return found;
    }
}
```

This is the same template — "mark visited" is the CHOOSE step, restoring the original character is the UN-CHOOSE.

10. Time & Space Complexity

Problem	Time	Space
All Subsets	O(2^n × n)	O(n) recursion
All Permutations	O(n! × n)	O(n)
N-Queens	O(n!) worst case	O(n)
Sudoku	O(9^k), k = empties	O(k) recursion
Rat in Maze	O(2^(n²)) worst	O(n²) for visited

Backtracking solutions are usually EXPONENTIAL in the worst case — that's the cost of exploring many possibilities. But pruning often cuts the practical runtime dramatically. The art of backtracking is finding GOOD pruning conditions.

11. Common Mistakes

Mistake 1: Forgetting to un-choose

```
current.add(arr[i]);
backtrack(...);
// ✗ forgot to: current.remove(current.size() - 1);
// Subsequent iterations will see polluted state
```

Mistake 2: Modifying input without restoring

```
// Word Search — without restoring:
grid[r][c] = '#';
boolean found = dfs(...);
return found;                    // ✗ grid is permanently corrupted

// Fix: save and restore
char saved = grid[r][c];
grid[r][c] = '#';
boolean found = dfs(...);
grid[r][c] = saved;              // ✓ restore
```

Mistake 3: Sharing a list across recursive calls without copying

```
if (isComplete()) {
    results.add(current);        // ✗ adds REFERENCE to mutable list
    return;
}
// All saved results are now THE SAME list!
```

```
if (isComplete()) {
    results.add(new ArrayList<>(current));    // ✓ snapshot
    return;
}
```

Mistake 4: Missing the base case

Without a base case, recursion never returns. The function keeps trying to make choices on a complete or invalid state. Always handle the "we're done" condition FIRST.

Mistake 5: Insufficient pruning

If you only check validity AT THE END, you've built a brute force, not a backtracker. The whole point is to fail FAST — check at every step whether the current partial solution is still valid.

Mistake 6: Wrong direction of recursion

In subset enumeration, the choice is "include or exclude this element." In permutation, the choice is "pick the next position." Mixing these up gives wrong answers. Be clear: WHAT are you choosing at each step?

12. When to Use Backtracking

Use backtracking when:

- The problem asks for ALL solutions, not just one ("all subsets," "all permutations")
- The problem asks if A solution EXISTS that satisfies constraints (N-Queens, Sudoku)
- Each step involves a CHOICE among several options
- Choices CAN BE UNDONE cleanly
- You can detect early if a partial choice is bad

Don't use backtracking when:

- There's a clear greedy or DP solution (much faster)
- You only need ONE answer and choices are independent
- The state can't be easily restored after a recursive call

13. Quick Reference

Concept	Meaning
State	Data representing "where I am" right now
Choices	All possible options at the current step
Constraint	Rule that filters valid from invalid choices

Concept	Meaning
Pruning	Skipping choices early when they can't lead to a solution
CHOOSE	Apply a tentative decision to the state
EXPLORE	Recurse into the next step
UN-CHOOSE	Reverse the decision so other options can be tried
Solution	A state that satisfies all constraints
Decision tree	The conceptual tree of all possible sequences of choices
DFS	Depth-first search — backtracking is DFS with un-choose

14. Practice Problems

Try these in order — each one teaches a different backtracking pattern.

Warm-ups

6. Print all subsets of an array (the power set). Both include/exclude and grow-the-list styles.
7. Print all permutations of a string or array.
8. Generate all binary strings of length N.
9. Print all combinations of choosing K elements out of N.

Grid Problems

10. Rat in a Maze: find all paths from top-left to bottom-right (moving down/right).
11. Rat in a Maze with 4 directions allowed (up/down/left/right) — track visited cells.
12. Word Search: check if a word can be traced in a 2D grid of letters.
13. Word Search II: find all words from a dictionary in the grid.
14. Knight's Tour: visit every square on an N×N board exactly once.

Classic Puzzles

15. N-Queens: print all valid placements for N=4, N=8.
16. Sudoku Solver: solve a 9×9 puzzle.
17. Tower of Hanoi: move N disks following the rules.

Interview Favorites

18. Generate all valid balanced parentheses of N pairs.
19. Letter combinations of a phone number (digits 2-9 → letters).

20. Combination Sum: find all combos summing to a target (repeats allowed).
21. Palindrome Partitioning: split a string so every substring is a palindrome.
22. Restore IP Addresses: given a digit string, return all valid IP combinations.
23. All paths from source to target in a DAG.

15. Final Takeaway

Backtracking is the universal tool for exploration problems. Once you see the CHOOSE → EXPLORE → UN-CHOOSE rhythm, dozens of problems that looked impossible become "oh, that's backtracking." Permutations, subsets, N-Queens, Sudoku, mazes, word search — same shape, different details.

The skill isn't memorizing the problems — it's recognizing the pattern. Ask yourself: "Am I making a sequence of decisions? Can I check each one as I go? Can I undo a decision?" If yes to all three, backtracking is your tool.

> **Key Takeaway:** Master the 5-step template: STATE → BASE CASE → CHOICES → PRUNING → CHOOSE/EXPLORE/UN-CHOOSE. Practice with the four core patterns: subsets, permutations, N-Queens, and grid search. Those four cover 80% of backtracking interviews. Always remember: every CHOOSE needs an UN-CHOOSE. Prune aggressively — that's where the speed comes from.

16. What's Next?

Backtracking is one of the last major DSA building blocks. From here:

- Dynamic Programming — memoization and tabulation for overlapping subproblems
- Greedy Algorithms — make the locally best choice (when it works)
- Heaps & Priority Queues — top-K problems, Dijkstra, scheduling
- Graphs — DFS, BFS, shortest path, topological sort
- Tries — prefix trees for autocomplete and word search optimization
- Bit Manipulation — alternative way to generate subsets (each bit = include/exclude)
- Sliding Window — efficient subarray and substring patterns